

自主學習：從 Python 到 APCS 三級

從零開始的演算法訓練

國三接觸程式後，我從 Python 的基礎語法開始練習，升上高中後逐步推進到 C 與 C++。基礎成熟後開始大量練習資料結構與演算法題目，累計完成超過兩百道練習題。

```
> ls
└─ apcs      └─ apcs10  └─ apcs12  └─ apcs14  ── consecutive.cpp
└─ homework      ── root.cpp    ── toj106.cpp  ── toj110.cpp
└─ apcs09  └─ apcs11  └─ apcs13  ── C09.cpp    ── guess.cpp
└─ oneonone.cpp  ── toj104.cpp  ── toj107.cpp  ── toj117.cpp
> fd -e .cpp | wc -l
205
```

自學方法與突破過程

▲各種演算法與基礎語法練習

這段自學歷程從零散的網路教學起步，經歷過看不懂題意就硬讀、寫不出解法就逐行比對、提交失敗時就上網拆解別人的思路，最終在 APCS 檢測中取得觀念題 4 級、實作題 3 級。這份成績的意義不在級數本身，而在於它記錄了我從「看不懂 for 迴圈」到「能獨力拆解中等難度問題」的整段路徑。

技術能力的轉化

擔任班級資訊股長期間，我也將自學中建立的除錯思維應用在協助師長與同學排解電腦及軟體問題上，把個人累積的技術能力轉化為班級的日常支援。

團隊實戰：pygame 專案與電算社

專案背景

高一加入電算社，隔年寒假參與社團靜態展製作。當時我們團隊被要求以 pygame 為 framework 開發一款小型遊戲作為社團代表作品。我擔任組長並主導整體架構設計與核心程式開發，組員負責測試與除錯回饋。

團隊協作的真正挑戰

過程中最大的挑戰不是技術本身，而是團隊協作。成員對 git 協作流程不熟悉，初期便出現權限設定錯誤與程式碼衝突。因此，我發現要使團隊合作成功必須更加注重組內的溝通才能避免衝突。

建立分工流程

於是在開發時，我先建立統一的分支規範，再將任務拆分為可獨立測試的模組，最後在會議上將這些模組一一分配給各組員並且完整說明。這樣的 **workflow** 不但使組內的噪音大幅降低，還使這次的 **project** 更快地完成。

帶走的觀念

這段經驗讓我理解到，真正的團隊開發能力不只是寫程式，更是把工程流程拆解到每個人都能參與的程度。後來，無論是變數命名、註解、**commit message** 都以「別人讀得懂」為優先。這樣，才能將自己的體悟，運用在未來的程式開發當中。

跨領域能力與總結

英語作為工程工具

在技術學習之外，我也重視語言作為工具的實際應用。我在未額外準備的情況下取得 **TOEIC 960** 分，這項能力源於長期閱讀英文技術文件與開源社群討論的習慣，而非刻意的考試準備。

三者交集

技術能力、團隊協作經驗與英語溝通能力三者的交集，構成了我進入大學後持續發展的基礎。我希望繼續做一個能獨立解決問題、也能流暢地將解決方法分享出去的人。